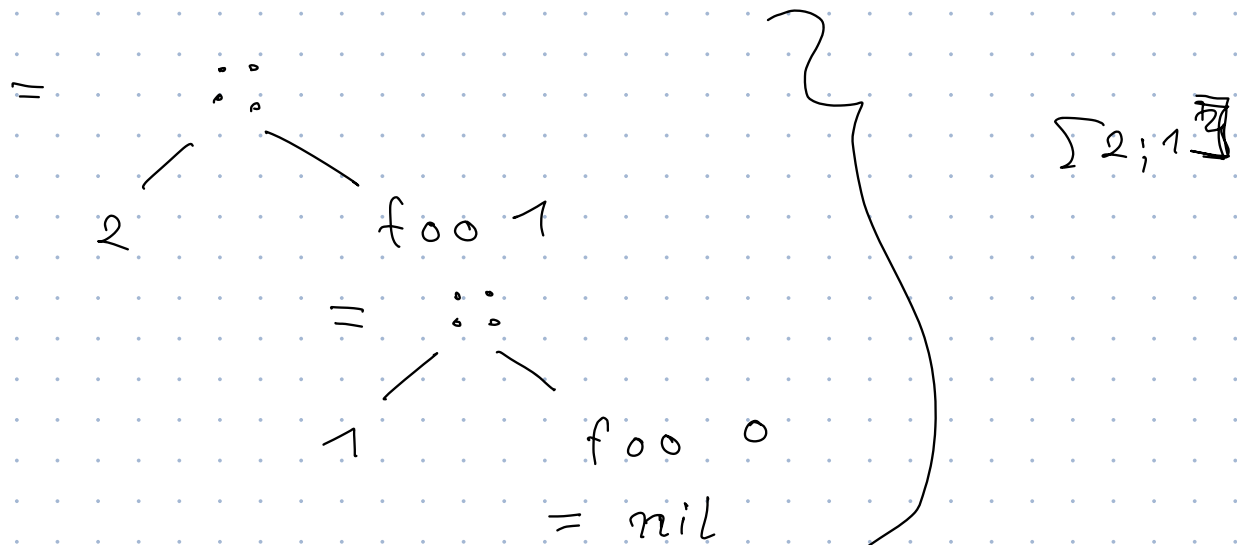


foo 2



To show:

$$\forall l_1 l_2 l_3. (l_1 ++ l_2) ++ l_3 = l_1 ++ (l_2 ++ l_3)$$

Proof by induction on l_1

case $l_1 = []$ i.s. $\underbrace{([] ++ l_2)}_{l_2} ++ l_3 = \underbrace{[] ++ (l_2 ++ l_3)}_{l_2 ++ l_3} \leftarrow$

This holds as $([] ++ l_2) ++ l_3$
 $= l_2 ++ l_3$ by Def ++
 $= [] ++ (l_2 ++ l_3)$ by Def ++

case $l_1 = h :: l_1'$ with $(l_1' ++ l_2) ++ l_3 = l_1' ++ (l_2 ++ l_3)$ (I.H).
 i.s. $\underbrace{((h :: l_1') ++ l_2) ++ l_3}_{\underbrace{(h :: (l_1' ++ l_2)) ++ l_3}_{h :: ((l_1' ++ l_2) ++ l_3)}} = \underbrace{(h :: l_1') ++ (l_2 ++ l_3)}_{h :: (l_1' ++ (l_2 ++ l_3))} \leftarrow$

This holds as
 $((h :: l_1') ++ l_2) ++ l_3 \stackrel{\text{by ++}}{=} h :: ((l_1' ++ l_2) ++ l_3) \stackrel{\text{I.H}}{=} h :: (l_1' ++ (l_2 ++ l_3))$
 $= (h :: l_1') ++ (l_2 ++ l_3)$

```
Fixpoint app (l1 l2 : natlist) : natlist :=
  match l1 with
  → | nil      => l2
  → | h :: t   => h :: (app t l2)
  end.
```

To show:

$$\forall c n. \text{repeat } n \text{ } c \text{ ++ repeat } n \text{ } b = \text{repeat } n \text{ } (c + b)$$

Proof by induction on c

Case $c = 0$ ✓

case $c = S c'$

with $\left[\text{repeat } n \text{ } c' \text{ ++ repeat } n \text{ } b = \text{repeat } n \text{ } (c' + b) \right]$

$$\begin{aligned} \text{I.S.} \quad & \underbrace{\text{repeat } n \text{ } (S c')}_{n :: \text{repeat } n \text{ } c'} \text{ ++ } \underbrace{\text{repeat } n \text{ } (S c')}_{n :: \text{repeat } n \text{ } c'} = \underbrace{\text{repeat } n \text{ } (S c' + S c')}_{S (c' + S c')} \\ & n :: (\text{repeat } n \text{ } c' \text{ ++ } (n :: \text{repeat } n \text{ } c')) \quad n :: \text{repeat } n \text{ } (c' + S c') \end{aligned}$$

$$\begin{aligned} \text{I.S.} \quad & \underbrace{\text{repeat } n \text{ } (S c')}_{(n :: \text{repeat } n \text{ } c')} \text{ ++ repeat } n \text{ } b = \underbrace{\text{repeat } n \text{ } (S c' + b)}_{S (c' + b)} \\ & n :: (\text{repeat } n \text{ } c' \text{ ++ repeat } n \text{ } b) \quad \begin{aligned} & n :: \text{repeat } n \text{ } (c' + b) \\ & \text{I.H.} \quad \text{repeat } n \text{ } c' \text{ ++ repeat } n \text{ } b \end{aligned} \end{aligned}$$

```
Fixpoint repeat (n count : nat) : natlist :=
  match count with
  | 0 => nil
  | S count' => n :: (repeat n count')
  end.
```

```
Fixpoint app (l1 l2 : natlist) : natlist :=
  match l1 with
  | nil => l2
  | h :: t => h :: (app t l2)
  end.
```